Cojocaru Dragos
Bicoiu Ionut

Documentatie proiect: Simulare N-Body

(Secvential / Pthreads / CUDA a/ MPI)

Rezumat

Acest proiect implementeaza o simulare N-Body cu interactii gravitationale intre particule. Au fost dezvoltate mai multe variante: CPU de baza (referinta), paralelizare cu Pthreads, accelerare pe GPU cu CUDA si o varianta distribuita cu MPI.

Rezultatele arata ca Pthreads obtine un speedup moderat (maxim ~2.9x), in timp ce CUDA obtine accelerare masiva (pana la ~121x) pentru acelasi set de intrari.

Descrierea problemei

Problema N-Body consta in simularea evolutiei unui sistem de N particule care interactioneaza prin forte gravitationale. Pentru fiecare pas de timp, se calculeaza forta rezultanta asupra fiecarei particule si se actualizeaza viteza si pozitia acesteia.

Aplicatii: astrofizica (simularea galaxiilor), dinamica moleculara, jocuri/efecte vizuale, simulari fizice in HPC.

Complexitate: implementarea directa are cost O(N^2) per pas de timp, deoarece fiecare particula interactioneaza cu toate celelalte.

Model matematic si algoritm

Pentru fiecare particula i se calculeaza suma contributiilor tuturor particulelor j != i. Fortele sunt determinate de legea gravitatiei universale (constanta G), iar integratorul folosit este de tip Euler explicit (actualizare viteza, apoi pozitie).

```
for step = 1..STEPS:
  for each i in [0..N-1]:
    (fx,fy,fz) = sum_{j!=i} F(i,j)
    v[i] += dt * f/m[i]
  for each i in [0..N-1]:
    x[i] += dt * v[i]
```

Necesitatea paralelizarii si profilare

Pentru N mare, bucla dubla de calcul al fortelor domina timpul total de executie.

Profilarea (ex. KCachegrind) evidentiaza ca cea mai costisitoare zona este calculul interactiunilor i-j (sqrt, operatii floating-point, acces repetat la memorie). Prin urmare, paralelizarea acestei bucle aduce cel mai mare castig.

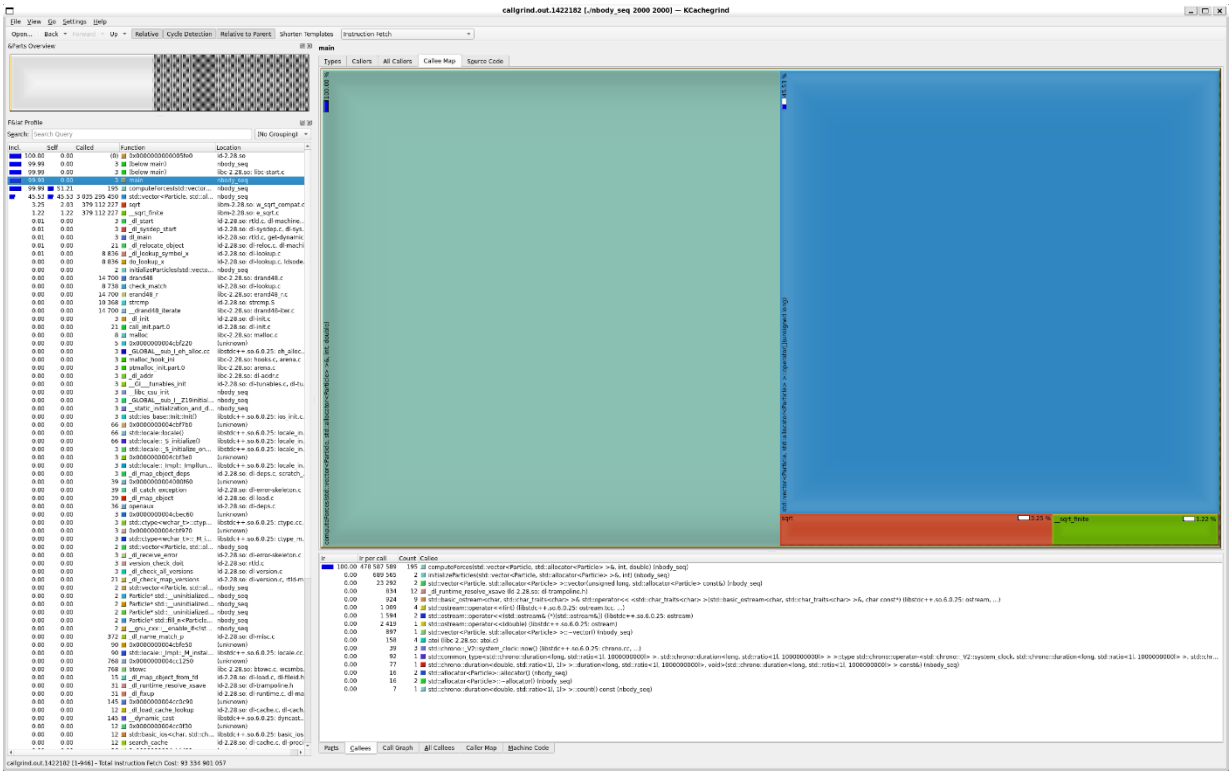

Figura 1 - Profilare (varianta Basic / referinta)

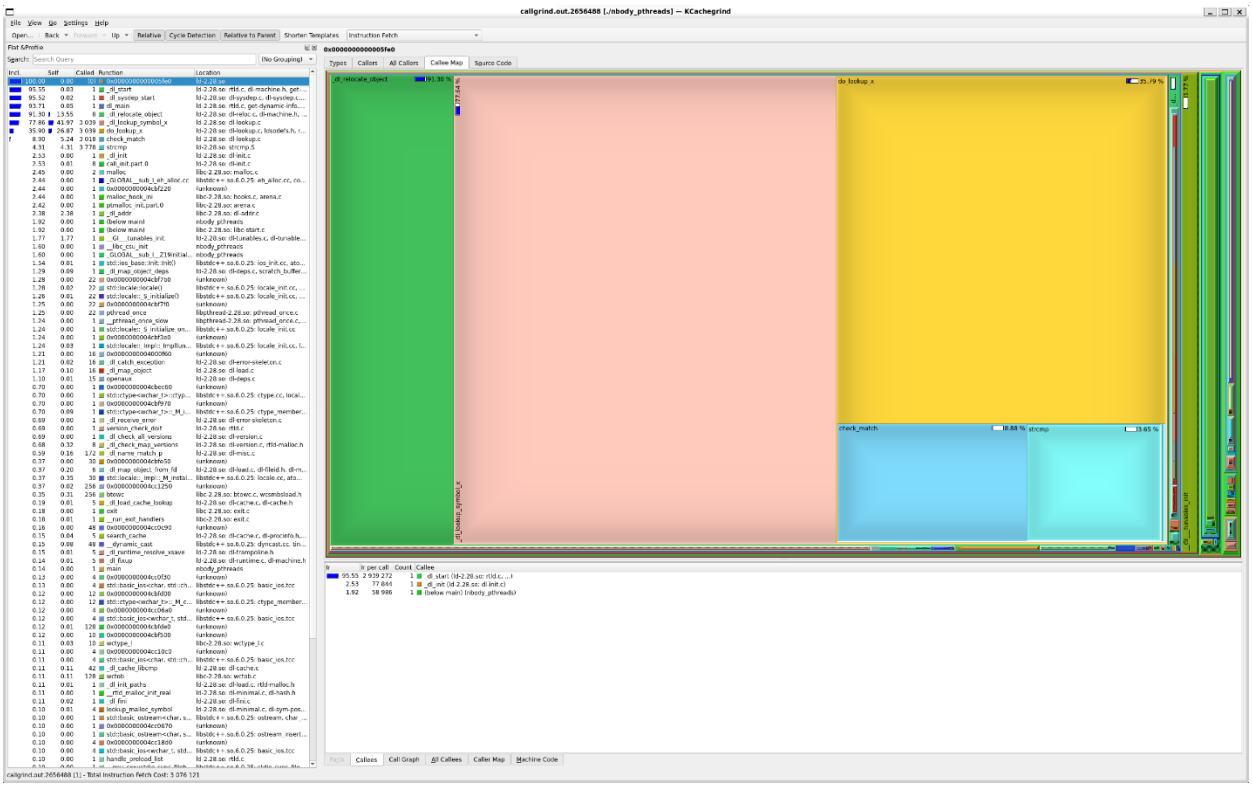

Figura 2 - Profilare (varianta Pthreads)

Implementari

Varianta de baza (CPU)

Varianta de referinta ruleaza pe CPU si executa integral calculul O(N^2) la fiecare pas. In masuratorile de performanta, timpul secvential de baza a fost: **223.815 s**.

Paralelizare cu Pthreads

In varianta Pthreads, particulele sunt impartite in intervale [start, end) pentru fiecare thread. Sunt paralelizate doua etape: (1) calculul fortelor si actualizarea vitezelor, (2) actualizarea pozitiilor. Implementarea foloseste pthread_create/pthread_join pentru fiecare pas de timp; acest lucru introduce un overhead semnificativ cand numarul de pasi este mare.

```
// impartire N in THREADS bucati (aproape egale)
pthread_create(..., computeForcesThread, &targs;[t]);
pthread_join(...);

// apoi, separat, updatePositionsThread pe aceleasi intervale
```

Accelerare cu CUDA (GPU)

Varianta CUDA lanseaza kernel-uri GPU care proceseaza particulele in paralel. Maparea folosita este: **1 thread GPU -> 1 particula i**, iar fiecare thread itereaza peste toate particulele j pentru a acumula forta rezultanta. Pentru performanta, sunt masurate doar timpii kernel-urilor (cu cudaEvent), iar initial se ruleaza un warm-up.

```
__global__ void computeForcesCUDA(Particle* p, int N, double dt){
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if(i >= N) return;
  // bucla peste j, acumulare forta
}
computeForcesCUDA<<>>(d_particles, N, dt);
updatePositionsCUDA<<>>(d_particles, N, dt);
```

Configurarea blocurilor este controlata din linia de comanda. In mod uzual, se poate folosi threads=256 si blocks=ceil(N/threads) sau un numar fix de blocks care satureaza GPU-ul.

Varianta distribuita cu MPI

In varianta MPI, particulele sunt distribuite intre procese (rank-uri) folosind un interval [start,end) calculat in functie de rank si size. La fiecare pas, fiecare proces calculeaza fortele pentru particulele locale, apoi se sincronizeaza starea globala printr-o operatie de comunicare colectiva (Allgather).

Nota: In implementarea curenta, pentru N care nu este multiplu de numarul de procese, Allgather ar necesita Allgatherv (dimensiuni variabile pe proces).

Experimente si rezultate

Timpul secvential de referinta: 223.815 s.

Masuratorile pentru Pthreads si CUDA au fost efectuate pentru un set de configuratii (4, 8, 12, 20, 24). Speedup-ul este definit ca T_seq / T_par, iar eficienta ca Speedup / P.

Configuratie	Timp (s)	Speedup	Eficienta
Basic (seq)	223.815	1.00	1.00
Pthreads (4 th)	215.447	1.04	0.26
Pthreads (8 th)	156.405	1.43	0.18
Pthreads (12 th)	77.302	2.90	0.24
Pthreads (20 th)	90.481	2.47	0.12
Pthreads (24 th)	104.024	2.15	0.09
CUDA (4 th)	8.250	27.13	6.78
CUDA (8 th)	4.450	50.30	6.29
CUDA (12 th)	3.100	72.20	6.02
CUDA (20 th)	2.050	109.18	5.46
CUDA (24 th)	1.850	120.98	5.04

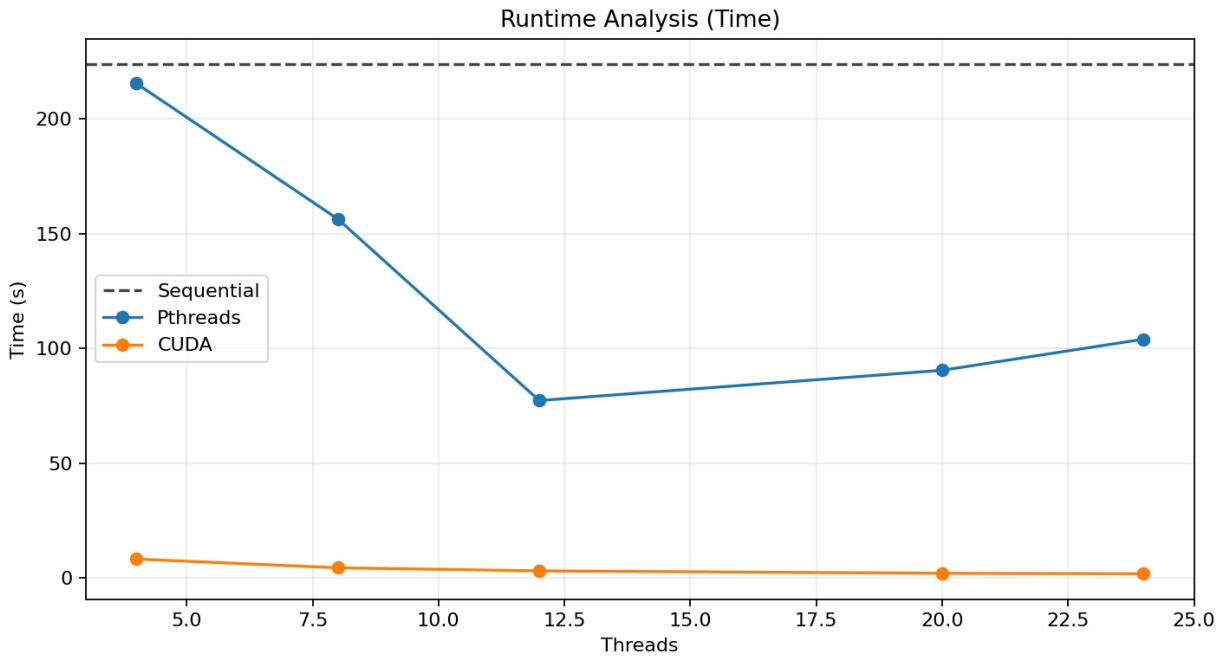

Figura 3 - Speedup (Pthreads vs CUDA)

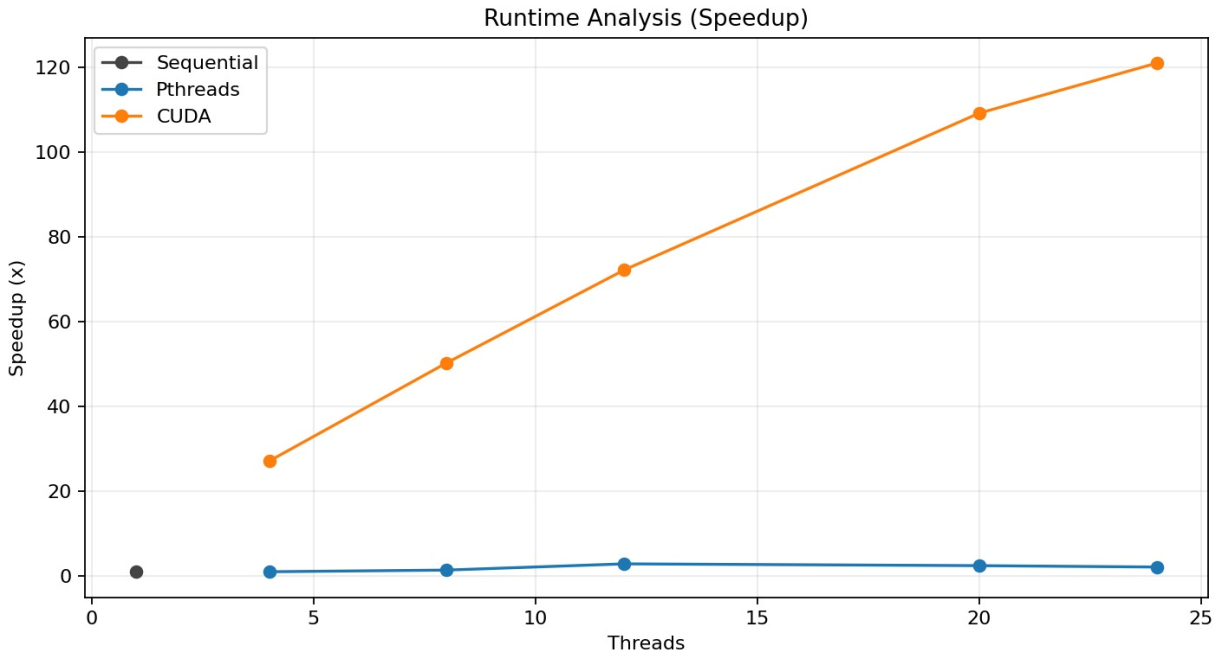

Figura 4 - Eficienta (Pthreads vs CUDA)

Discutii

Pthreads obtine un castig moderat deoarece CPU-ul are un numar limitat de nuclee, iar performanta este constransa de bandwidth-ul memoriei si de overhead-ul de sincronizare. Un varf de performanta este observat la 12 thread-uri (~2.9x), dupa care cresterea numarului de thread-uri nu mai ajuta.

CUDA obtine accelerare masiva deoarece GPU-ul poate executa in paralel un numar foarte mare de thread-uri. Modelul SIMT este potrivit pentru N-Body, unde fiecare particula este procesata independent. Eficienta aparent > 1 apare deoarece raportarea la 'threads' nu reflecta echivalentul real de paralelism hardware (warp-uri, SM-uri, latenta ascunsa).

Concluzii

Secvential (referinta): 223.815 s.

Pthreads: speedup maxim ~2.9x (12 thread-uri), eficienta maxima ~0.26.

CUDA: speedup maxim ~121x, timp minim ~1.85 s.

Pentru N-Body, accelerarea pe GPU este abordarea dominanta din punct de vedere al performantei. Pthreads ramane o solutie portabila si relativ simpla, dar este limitata de arhitectura CPU si overhead.